

Types of Operators in Java :

- Basically there are 3 types of operators :
- a) Unary Operator
 - b) Binary Operator
 - c) Ternary Operator

Operator Diagram with Priority :

Operator Precedence	
Operators	Precedence
postfix	<code>expr++ expr--</code>
unary	<code>++expr --expr +expr -expr ~ !</code>
multiplicative	<code>* / %</code>
additive	<code>+ -</code>
shift	<code><< >> >>></code>
relational	<code>< > <= >= instanceof</code>
equality	<code>== !=</code>
bitwise AND	<code>&</code>
bitwise exclusive OR	<code>^</code>
bitwise inclusive OR	<code> </code>
logical AND	<code>&&</code>
logical OR	<code> </code>
ternary	<code>? :</code>
assignment	<code>= += -= *= /= %= &= ^= = <<= >>= >>>=</code>

Operator Precedence:

In order to calculate the result, we need to know Operator precedence. It describes operators are evaluated in what order.

Arithmetic Operator OR Binary Operator:

It is known as Arithmetic Operator OR Binary Operator because it works with minimum two operands.

Ex:- `+`, `-`, `*`, `/` and `%` (Modula Or Modulus Operator)

```
Optry1.java
-----
void main()
{
    int height = 3;
    int length = 5;

    var perimeter = 2 * height + 2 * length; //16
    IO.println(perimeter);
}
```

Note : The multiplication Operator has highest precedence than addition.

```
Optry2.java
-----
void main()
{
    int height = 3;
    int length = 5;

    var perimeter = 2 * (height + 2) * length; //2 * 5 * 5
    IO.println(perimeter);
}
```

Note: We can provide priority OR change the operator precedence by using `()` parentheses. If two operators are same precedence than java will evaluate **left to right**.

Working with modulus Operator :

WAP to find the reverse of a 3 digit number

```
Optry3.java
-----
void main()
{
    int number = Integer.parseInt(IO.readLine("Enter a Number : ")); //567 -> 765

    int rem = number % 10; // rem = 7
    IO.print("Reverse of 3 digit number is :"+rem); //7

    number = number / 10; //number = 56

    rem = number % 10; // rem = 6

    IO.print(rem);

    number = number / 10; //number = 5

    IO.print(number);
    IO.println();
}
```

Unary Operators:

The operator which works upon **single operand** is called Unary Operator. In java we have so many unary operators as shown in the Operator diagram.

- a) `!` : It is logical complement Operator. It will invert the boolean value.
- b) `+` : It is unary `+` operator. [by default, number is positive only]
- c) `-` : It is unary `-` Operator.

d) `++` : It is unary increment operator, Increments the value by 1

e) `--` : It is unary decrement operator, Decrements the value by 1

```
Optry4.java
-----
void main()
{
    boolean isEligible = true;
    IO.println(isEligible);
    IO.println(!isEligible);
}
```

```
Optry5.java
-----
void main()
{
    int x = 15;
    IO.println(-x); //Unary minus Operator
    IO.println(-(-x));
}
```

What is numeric promotion in Java :

While working with **Arithmetic Operator (+,-,*,/,%) OR Unary minus operator** the smaller data type values are automatically promoted to bigger data type to prevent from data loss.

Numeric Promotion Chart :

	Operand1	Operand2	Promoted Type (Result)	
	byte	byte	int	byte, short and char first promoted to int type
	byte	short	int	
	short	short	int	
	char	char	int	
	int	long	long	
	long	float	float	
	float	double	double	

```
Optry6.java
-----
void main()
{
    byte b = 125;
    byte c = 125;
    byte d = b + c; //error [Promoted to int type]
    IO.println(d);
}
```

```
Optry7.java
-----
void main()
{
    short s = 12000;
    short t = 12000;
    short u = s + t; //error [Promoted to int type]
    IO.println(u);
}
```

```
Optry8.java
-----
void main()
{
    byte b = 1;
    byte c = -b; //error [Promoted to int type]
    IO.println(c);
}
```

```
Optry9.java
-----
void main()
{
    byte b = 1;
    byte c = --b; //valid
    IO.println(c); //0
}
```

```
Optry10.java
-----
void main()
{
    byte b = 1;
    b = b + 5; //error
    b += 5; //valid [Here javac will convert into byte type explicitly]
    IO.println(b);
}
```

Working with Increment & Decrement Operator :

Initial Value of x	Expression	Value of y	After evaluation final value of x
int x = 10;	int y = ++x; //Pre Increment	11	11
int x = 10;	int y = x++; //Post Increment [First Assignemnt then Increment]	10	11
int x = 10;	int y = --x; //Pre Decrement	9	9
int x = 10;	int y = x--; //Post Decrement First Assignemnt then Decrement	10	9

```
Optry11.java
-----
void main()
{
    int x = 15;
    int y = ++x; //First increment then assignment
    IO.println(x+"-"+y);
}
```

`x++` `--` Will use old value
`++x` `--x` Will use new value

```
Optry12.java
-----
void main()
{
    int x = 15;
    int y = x++; //First Assignment then Increment
    IO.println(x+"-"+y);
}
```

```
Optry13.java
-----
void main()
{
    int x = 15;
    int y = ++x+5; //Unexpected type error
    IO.println(y);
}
```

```
Optry14.java
-----
void main()
{
    int x = 15;
    int y = ++(++x); //Unexpected type
    IO.println(y);
}
```

Increment & decrement Operator we cannot apply on constants, only for variables

```
Optry15.java
-----
void main()
{
    char ch ='A';
    ch++;
    IO.println(ch);
}
```

```
Optry16.java
-----
void main()
{
    double d = 15.15;
    d++;
    IO.println(d);
}
```

Note :- Increment and decrement operator we can apply with any data type except boolean.

```
Optry17.java
-----
void main()
{
    int amount = 0;
    IO.println(amount); //0
    IO.println(++amount); //1
    IO.println(amount); //1
    IO.println(amount--); //1
    IO.println(amount); //0
}
```

Increment & decrement operator is having high order of precedence as compared to binary operator.

```
Optry18.java
-----
void main()
{
    int i = 5; //i = 6
    int j = i++ + 4; //5 + 4
    IO.println(i+" : "+j);
}
```

```
Optry19.java
-----
void main()
{
    int i = 5; //6
    int j = ++i + 4; //6 + 4
    IO.println(i+" : "+j);
}
```

Note : `i++` always uses old value but `++i` will use the new value same rule for `i--` & `--i`

```
void main()
{
    int i = 5;
    int j = --i - 4;
    IO.println(i+" : "+j);
}
```

```
IQ
void main()
{
    int i = 5;
    int result = i++ + ++i;

    IO.println("i = " + i);
    IO.println("result = " + result);
}
```

```
IQ
void main()
{
    int a = 5;
    int b = a++ + a++;

    IO.println(a);
    IO.println(b);
}
```

```
void main()
{
    int x = 10;
    int y = ++x + x++;

    IO.println(x);
    IO.println(".....");

    int y = 15;
    IO.println(++y + ++y);
    IO.println(y);
}
```

```
void main()
{
    int x = 3;
    int y = ++x * 5 / x--;
    IO.println(x+" : "+y);
}
```

```
void main()
{
    int a = 4;
    int b = a++ * 2 / --a;
    IO.println(a+" : "+b);
}
```

```
void main()
{
    int a = 6; //7
    int b = a++ + a-- + ++a;
    IO.println(a+" : "+b);
}
```

Relational Operator :-

These operators are used to compare the values. The return type is **boolean**.

- 1) `>` (Greater than)
- 2) `<` (Less than)
- 3) `>=` (Greater than or equal to)
- 4) `<=` (Less than or equal to)
- 5) `==` (double equal to)
- 6) `!=` (Not equal to)
- 7) `instanceof` operator [We will learn in Polymorphism concept]